Repetition Structures

- print "hello" ten times

```
print("hello")
print("hello")
print("hello")
  :
print("hello")
```

} repeat 10 times

want to be able to repeat easily

Two loops

1) Loop while the condition is True : while
2) Loop through a list of items : For loop

---

while structure

- Keep looping while the condition is True
  ↳ Out of the loop when the condition is False
- Syntax:

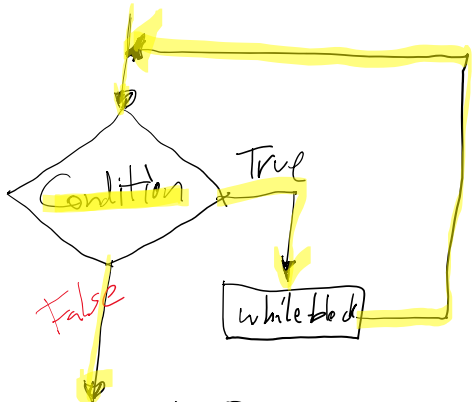
```
while condition:
    statement
    statement
```

True/False

True → while-block = things to be repeated

False

Space to indent inside while-block



* If the condition is never False, you stuck in the loop.

you stuck in the loop!

EX

```
x = 0
while x < 4:
    print(x)
    x = x+1
print("Done")
```

while-block

outside while

Output:
0
1
2
3
Done

while block

x

0 1

x = 0

loop 1
x < 4 → 0 < 4 → True
print(x) → print(0) → 0
x = x+1 → x = 0+1 = 1

loop 2
x < 4 → 1 < 4 → True
print(x) → print(1) → 1
x = x+1 → x = 1+1 = 2

loop 3
x < 4 → 2 < 4 → True
print(x) → print(2) → 2
x = x+1 → x = 2+1 = 3

loop 4
x < 4 → 3 < 4 → True
print(x) → print(3) → 3
x = x+1 → x = 3+1 = 4

loop 5
x < 4 → 4 < 4 → False

out of the loop!

print("Done") → Done

EX

```
x = 5
while x >= 0:
    print(x)
    x = x-1
print("Done")
```

Output:
5
4
3
2
1
0
Done

#1
5 >= 0 ✓
print(5)
x = 5-1 = 4

#2
4 >= 0 ✓
print(4)
x = 4-1 = 3

#3
3 >= 0 ✓
print(3)
x = 3-1 = 2

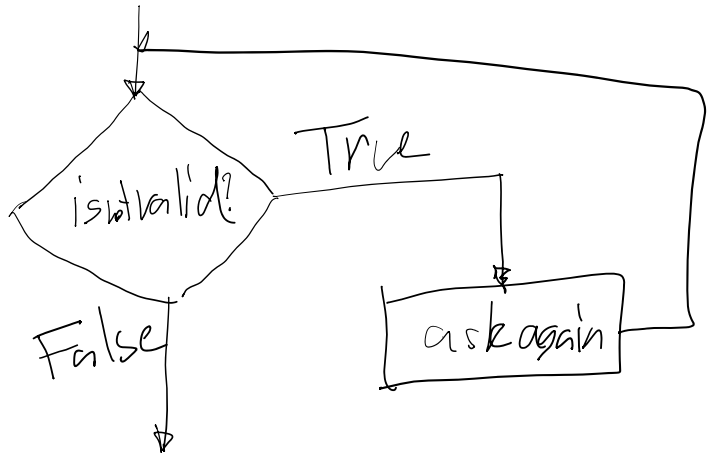
#7
-2 >= 0 ✗ → out

-1
0
Done

Input Validation

↳ Check that inputs are valid
  ↳ e.g. age is non-negative
  e.g. height should be between 0 and 300 cm.
↳ Ask user to type in again until their input is valid.
  ↳ keep asking → keep looping

out of the loop!



Ex Validate positive age.

```
age = int(input("Enter age"))
while age <= 0:          # check not valid
    print("Age must be positive")
    age = int(input("Enter age: "))
print(age)
```

the first loop, age has no value

Ex2 Calculate area based width and length

EX2 Calculate area based width and length

must be positive

Before

```
width = float(input())
length = float(input())
area = width * length
print(area)
```

validate

After

```
width = float(input())
while width <= 0:
    width = float(input())
# get out → width is positive
length = float(input())
while length <= 0:
    length = float(input())
area = width * length
print(area)
```

List

- is a collection of items (data type)
- Syntax: [item1, item2, item3, item4]

e.g. [1, 2, 3, 4, 5]

[True, "A", "ABC", 1, 1.5]

- Can store in a variable

```
x = [1, 2, 3]
print(x) → [1, 2, 3]
```

For loop

↳ loop through items in the list/collection of items from left to right one by one.

↳ Syntax:

```
for var in [item1, item2, item3]:
    statement
    statement
```

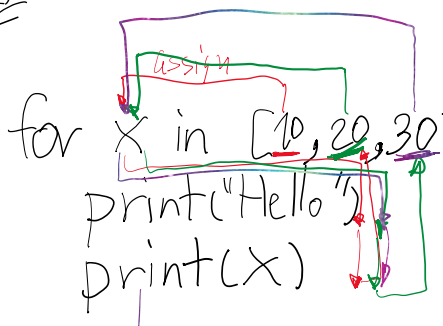
assign

for-block

get repeated

^ variable to store items in the list

Ex



Out

output:
Hello
10
Hello
20
Hello
30

```
# loop 1
x = 10
print("Hello")  → Hello
print(x)        → 10
# loop 2
x = 20
print("Hello")  → Hello
print(x)        → 20
# loop 3
x = 30
print("Hello")  → Hello
print(x)        → 30
```

Calculate the sum of numbers

- given a list of numbers, find the sum of numbers

  e.g. [10, 20, 30, 40] ⟶ 10+20+30+40=100

numbers

e.g. [10, 20, 30, 40] → 10+20+30+40=100

adding them to the total ← repeat!

```
numbers = [10, 20, 30, 40]
total = 0
for num in numbers:
    total = total + num
print(total)
```

$$total = \sum_{i=1}^{n} x_i$$

# loop 1
num = 10
total = 0 + 10 = 10

# loop 2
num = 20
total = 10 + 20 = 30

# loop 3
num = 30
total = 30 + 30 = 60

# loop 4
num = 40
total = 60 + 40 = 100

Counting things

↳ given a list of items, count the number of items satisfy the condition

↳ e.g. count the number of even numbers

[10, 9, 21, 22, 30, 4]
+1 ✗ ✗ +1 +1 +1 = 4

repeat: check that num is even?
↳ True add 1 to the count!

```
count = 0
seqs = [10, 9, 21, 22, 30, 4]
```

```
for num in seqs:
    if num % 2 == 0:
        count = count + 1
print(count)
```

forblock

update logic

Short-hand notation

	short	
`X = X + 1`	→	`X += 1`
`count = count + 1`	→	`count += 1`
`total = total + num`	→	`total += num`
`y = y - 5`	→	`y -= 5`
`z = z * 3`	→	`z *= 3`
`w = w / (5 - 2)`	→	`w /= (5 - 2)`

range function

↳ generate a sequence of numbers in the following pattern: start, start+step, start+2step, ... end

[0, 1, 2, 3, 4]

↳ Syntax: range(start, end, step)

step: can be negative

the sequence will end before the end

- The sequence will end before the end

Ex range(1, 5, 1) → [1, 2, 3, 4]
(end before)

range(5, 1, -1) → [5, 4, 3, 2]

range(7, 2, -2) → [7, 5, 3]

* stop when the next number is at or after the end

Short versions

- range(start, stop) → step = 1 (default)
  - ↳ e.g. range(1, 5) → range(1, 5, 1) → [1, 2, 3, 4]
  - range(2, 7) → range(2, 7, 1) → [2, 3, 4, 5, 6]
- range(stop) → start = 0, step = 1 (default)
  - ↳ e.g. range(4) → range(0, 4, 1) → [0, 1, 2, 3]

Ex

```
for x in range(3):   → [0, 1, 2]
    print(x)
```

Output
0
1
2

---

break statement

↳ whenever you encounter break,

↳ whenever you encounter break, you get out of the loop!
↑ both while & for

Ex
```
x=5
while x>=0:
    print(x)
    x=x-1
    break
print(x)
```

#1
```
5>=0
print(x) → 5
x=x-1 → 5-1=4
break → out of the loop!
print(x) → 4
```

Output 5
4

Ex
```
for i in [10, 20, 11, 22, 30]:
    print(i)
    if i%.2==1:
        break
```

10 → 20 → 11

11%.2==1 ✓

Output 10
20
11

Summary

- while : while condition is true
- for : loop through a list
- shorthand : x=x+1 → x+=1
- range(start, end, step)
  1 1

- range (start, end, step)
- break